CO5 AT2

Scenario-Based Assessment Answers

Name: Galla Sai Siddartha
Reg No: 192425093
Course: Reinforcement Learning
Course Code: MLA0302

Question 1: RL Framework for Personalized Mobile App Notifications

A mobile app can be modelled as a Markov Decision Process in which an agent (the notification engine) learns, from repeated interaction, when and what to notify a specific user so that engagement improves without causing fatigue or app uninstalls.

State Space

- Time of day and day of week
- User's recent in-app activity (session frequency, last active time, screens visited)
- Historical response to past notifications (opened, ignored, dismissed, disabled)
- User demographic and preference segment
- Device context such as battery level, location category, and connectivity

Action Space

- Send a notification now vs. wait
- Choose the content/category of the notification (offer, reminder, social update, news)
- Choose the channel or format (push banner, in-app card, badge)
- Choose notification frequency for the day

Reward Function

- Positive reward for a notification that is opened within a short time window
- Small positive reward for increased session length or a completed action after the notification
- Negative reward (penalty) when the notification is dismissed instantly or the user disables notifications
- A decaying penalty for sending too many notifications in a short period, to discourage spamming

Suitability

This is naturally sequential and personalized: the best notification policy for a user changes over time, which is exactly what RL, unlike a one-shot supervised classifier, is designed to optimize through long-term reward accumulation rather than a single prediction.

Illustrative Code (Python)

```
import random
class NotificationEnv:
    def __init__(self):
        self.actions = ["no_send", "send_offer", "send_reminder", "send_social"]
    def get_state(self, user):
        return (user["hour"], user["last_open_gap"], user["recent_ignores"])
    def reward(self, action, opened, session_gain, spam_flag):
        if action == "no_send":
            return 0
        r = 5 if opened else -1
        r += 0.1 * session_gain
        if spam_flag:
            r -= 3
        return r

# Simple tabular Q-learning update
Q = {}
```

```
def q_update(state, action, reward, next_state, alpha=0.1, gamma=0.9):
    Q.setdefault(state, {a: 0.0 for a in NotificationEnv().actions})
    Q.setdefault(next_state, {a: 0.0 for a in NotificationEnv().actions})
    best_next = max(Q[next_state].values())
    Q[state][action] += alpha * (reward + gamma * best_next - Q[state][action])
```

Question 2: Exploration vs. Exploitation in Warehouse Robot Navigation

The warehouse robot must balance exploitation, choosing the path it currently believes is shortest and obstacle-free, against exploration, trying alternative paths to discover whether an even better or safer route exists. If exploration is insufficient, the robot's policy converges prematurely to a locally optimal but globally suboptimal path, causing it to repeatedly take longer routes even when shorter ones exist.

Effect of Poor Exploration on Performance

- The agent gets stuck in a locally optimal policy learned early in training
- Newly opened or temporarily blocked routes are never re-discovered, so the map knowledge becomes stale
- Delivery time and energy consumption increase because the robot avoids under-tried but potentially shorter paths
- The Q-value estimates for rarely visited states remain inaccurate, reducing confidence in the learned policy

Suggested Improvements

- Use a decaying epsilon-greedy strategy so exploration is high early in training and gradually reduces as the policy matures
- Apply upper-confidence-bound (UCB) style exploration that favours states/actions with high uncertainty
- Use reward shaping that lightly penalizes excessively long paths so exploitation of good routes is reinforced faster
- Periodically inject forced exploration episodes or use curiosity-driven intrinsic rewards to revisit rarely explored areas
- Maintain an experience replay buffer so rare but informative transitions are reused efficiently instead of being forgotten

Illustrative Code (Python)

```
import random
def epsilon_greedy_action(Q, state, actions, epsilon):
    if random.random() < epsilon:
        return random.choice(actions)  # explore
    return max(actions, key=lambda a: Q[state].get(a, 0.0))  # exploit

def decay_epsilon(epoch, start=1.0, end=0.05, decay_rate=0.01):
    return end + (start - end) * (2.71828 ** (-decay_rate * epoch))

# Training loop sketch
epsilon = 1.0
for episode in range(1000):
    epsilon = decay_epsilon(episode)
    state = env_reset()
    done = False
    while not done:
        action = epsilon_greedy_action(Q, state, actions, epsilon)
        next_state, reward, done = env_step(state, action)
        q_update(state, action, reward, next_state)
        state = next_state
```

Question 3: RL for Hospital Appointment Scheduling

Suitability of RL

Hospital scheduling is a sequential decision-making problem: each scheduling decision affects future slot availability, doctor workload, and patient waiting time, which matches the MDP structure RL is built for. The agent can learn a scheduling policy that minimizes average waiting time and maximizes utilization of rooms, staff, and equipment by observing outcomes of past scheduling decisions and adjusting accordingly.

How RL Could Be Applied

- States: current queue length, doctor/resource availability, patient priority/urgency, time of day
- Actions: assign a patient to a particular time slot, doctor, or resource; or delay assignment
- Rewards: negative reward proportional to waiting time, positive reward for high resource utilization and on-time completion

Challenges

Delayed rewards: the true cost of a poor scheduling decision (e.g., a bottleneck) may only appear several hours later, making credit assignment difficult and requiring techniques such as eligibility traces or n-step returns.

Ethical considerations: the policy must not learn to systematically deprioritize certain patient groups (e.g., based on insurance type) purely because it improves a numeric reward; fairness constraints and human oversight are required, along with transparency in how urgent/emergency cases are always prioritized regardless of reward.

Data sparsity and safety: mistakes during training/exploration on a live scheduling system can directly harm patients, so simulation-based training or offline RL from historical data is preferable to online trial-and-error.

Illustrative Code (Python)

```
class SchedulingEnv:
    def __init__(self, slots, doctors):
        self.slots = slots
        self.doctors = doctors
    def step(self, patient, action_slot):
        wait_time = action_slot["time"] - patient["arrival_time"]
        utilization = self.compute_utilization()
        reward = -0.5 * wait_time + 2.0 * utilization
        if patient["urgent"] and wait_time > 15:
            reward -= 10  # ethical/priority penalty
        return reward
    def compute_utilization(self):
        used = sum(1 for s in self.slots if s["assigned"])
        return used / len(self.slots)

# n-step return to handle delayed rewards
def n_step_return(rewards, gamma=0.95):
    G = 0.0
    for i, r in enumerate(rewards):
        G += (gamma ** i) * r
    return G
```

Question 4: RL Model for Smart Home Energy Optimization

State Space

- Occupancy/presence detection per room, indoor temperature and humidity, outdoor weather forecast, time of day, and current device states (on/off, fan/AC speed)

Action Space

- Turn lights/fans/AC on or off
- Adjust AC thermostat set-point or fan speed level
- Pre-cool or delay operation based on predicted occupancy

Reward Structure

- Positive reward for reduced electricity consumption over a time window
- Positive reward for maintaining comfort (temperature within a user-preferred range) while occupied
- Negative reward/penalty for discomfort (temperature outside range while occupants are present)
- Negative reward for energy wasted while rooms are unoccupied

Suitable Learning Algorithm

Since the state and action spaces are largely continuous (temperature, fan speed) and the environment is only partially observable (predicting occupancy and weather), a Deep RL method such as Deep Q-Network (DQN) for discretized appliance control, or an actor-critic method such as Proximal Policy Optimization (PPO) or DDPG for continuous thermostat control, is well suited. A multi-objective reward (comfort vs. energy saving) also benefits from PPO's stable policy-gradient updates.

Illustrative Code (Python)

```
def compute_reward(power_kwh, temp, comfort_range, occupied):
    reward = -1.0 * power_kwh  # energy penalty
    low, high = comfort_range
    if occupied:
        if low <= temp <= high:
            reward += 2.0  # comfort bonus
        else:
            reward -= 3.0  # discomfort penalty
    else:
        if power_kwh > 0.1:
            reward -= 2.0  # wasted energy penalty
    return reward

# PPO-style actor-critic policy update sketch (pseudo)
def ppo_update(policy, value_fn, states, actions, advantages, returns, clip_eps=0.2):
    old_probs = policy.log_prob(states, actions).detach()
    new_probs = policy.log_prob(states, actions)
    ratio = (new_probs - old_probs).exp()
    clipped = ratio.clamp(1 - clip_eps, 1 + clip_eps)
    policy_loss = -min(ratio * advantages, clipped * advantages).mean()
    value_loss = ((value_fn(states) - returns) ** 2).mean()
    return policy_loss + 0.5 * value_loss
```

Question 5: Reward Design for RL-based Matchmaking

In skill-based matchmaking, the RL agent selects which players to group together, and the reward signal directly shapes what the system optimizes for. Reward design here is especially sensitive because a seemingly reasonable reward can create unintended, biased matching behaviour.

How Reward Design Influences Matchmaking Quality

- Rewarding purely for 'close skill rating' encourages tight, competitive matches, which typically improves short-term engagement and perceived fairness
- Rewarding purely for 'match speed' (minimizing queue time) encourages the agent to pair players quickly even with large skill gaps, hurting match quality

- Rewarding for session length or in-app purchases after a match can unintentionally encourage matching weaker players against stronger ones to boost engagement of paying users, which is unfair to others

Risks of Poorly Designed Rewards

- Biased matching toward a subset of high-value or highly active users, disadvantaging casual or new players
- Reward hacking, where the agent learns shortcuts (e.g., artificially inflating win/loss variance) that raise the numeric reward without genuinely improving fairness or fun
- Reduced retention among players who consistently receive imbalanced matches, even if aggregate reward looks good

Mitigation

Combine multiple sub-rewards (skill balance, queue time, retention) with carefully tuned weights, and audit outcomes across player segments rather than relying on a single aggregate engagement metric.

Illustrative Code (Python)

```
def matchmaking_reward(skill_gap, queue_time, retention_signal,
                       w_skill=2.0, w_queue=0.5, w_retention=1.0):
    skill_penalty = -w_skill * abs(skill_gap)
    queue_penalty = -w_queue * queue_time
    retention_bonus = w_retention * retention_signal
    return skill_penalty + queue_penalty + retention_bonus

def audit_fairness(match_log, segment_key):
    segment_rewards = {}
    for m in match_log:
        seg = m[segment_key]
        segment_rewards.setdefault(seg, []).append(m["reward"])
    return {seg: sum(v) / len(v) for seg, v in segment_rewards.items()}
```

Question 6: RL for Financial Trading (States, Actions, Rewards, Delayed-Reward Risk)

State Space

- Recent price history and technical indicators (moving averages, volatility, volume)
- Current portfolio position (holdings, cash balance, exposure)
- Broader market signals (news sentiment, macroeconomic indicators, order book depth)

Action Space

- Buy, sell, or hold a given asset
- Choose position size or leverage
- Set or adjust stop-loss / take-profit levels

Reward Function

- Realized profit or loss from a trade, often risk-adjusted using a metric such as the Sharpe ratio rather than raw return, so the agent is not rewarded for reckless high-variance strategies

Risks of Delayed Rewards in Volatile Markets

The true outcome of a buy/sell decision may only be known after the position is closed, sometimes days or weeks later, making credit assignment to individual earlier actions difficult (temporal credit assignment problem).

High market volatility increases variance in returns, so a delayed reward signal can be noisy and misleading, causing the agent to learn spurious correlations between an action and an outcome that was actually driven by unrelated market shocks.

Overfitting to a particular historical volatility regime is a real risk; techniques such as reward shaping with intermediate risk-based signals, n-step returns, and robust offline evaluation on multiple market regimes help reduce this risk before any live deployment.

Illustrative Code (Python)

```
import statistics
def sharpe_reward(returns, risk_free_rate=0.0):
    if len(returns) < 2:
        return 0.0
    excess = [r - risk_free_rate for r in returns]
    mean_r = sum(excess) / len(excess)
    std_r = statistics.pstdev(excess) or 1e-6
    return mean_r / std_r

def trading_action(state, policy):
    return policy.select_action(state)  # returns "buy" / "sell" / "hold"

# eligibility trace update for delayed-reward credit assignment
def eligibility_trace_update(Q, trace, state, action, reward, next_state,
                             alpha=0.1, gamma=0.95, lam=0.8):
    td_error = reward + gamma * max(Q[next_state].values()) - Q[state][action]
    trace[(state, action)] = trace.get((state, action), 0) + 1
    for (s, a), e in trace.items():
        Q[s][a] += alpha * td_error * e
        trace[(s, a)] = gamma * lam * e
```

Question 7: RL Framework for Drone Delivery Route Optimization

State Space

- Drone's current GPS position, altitude, battery level, wind/weather conditions, and a map of restricted airspace zones and known obstacles

Action Space

- Move in a chosen direction/altitude change
- Hold position or land
- Choose an alternate waypoint when a restricted zone is detected

Reward Function

- Positive reward for progress toward the delivery destination and for successful, on-time delivery
- Large negative penalty for entering restricted airspace or violating a minimum safe altitude
- Smaller negative penalty per time step or per unit of battery consumed, to encourage efficient routing

Justification for Penalties Ensuring Safe Navigation

Because restricted airspace violations carry severe real-world safety and legal consequences, they must be weighted far more heavily (a large negative penalty, potentially episode-terminating) than the small step-wise efficiency penalties. This reward asymmetry teaches the policy that safety constraints dominate over path-length optimization, so the agent will accept a longer but safe route rather than a short but restricted one. Combining this with constrained RL (hard action masking near restricted zones) provides an additional safety guarantee beyond what the reward signal alone can offer.

Illustrative Code (Python)

```
RESTRICTED_ZONE_PENALTY = -100
STEP_PENALTY = -0.1
BATTERY_PENALTY_PER_UNIT = -0.05

def drone_reward(distance_to_goal_delta, in_restricted_zone,
                 battery_used, delivered):
    reward = distance_to_goal_delta  # progress toward goal
    reward += STEP_PENALTY
    reward += BATTERY_PENALTY_PER_UNIT * battery_used
    if in_restricted_zone:
        reward += RESTRICTED_ZONE_PENALTY
    if delivered:
        reward += 50
    return reward

def valid_actions(state, restricted_zones, actions):
    # action masking: remove actions that lead directly into restricted airspace
    return [a for a in actions if not enters_zone(state, a, restricted_zones)]
```

Question 8: RL for Personalized Online Learning Paths

How Reward Design Influences Student Engagement

- Rewarding the agent for content completion rate can push it toward recommending easier material that students finish quickly, rather than material that actually builds mastery
- Rewarding for quiz/assessment performance encourages recommending appropriately challenging content that reinforces weak concepts, which tends to improve genuine learning outcomes
- Rewarding purely for time spent on the platform can lead to recommending repetitive or overly long content that maximizes engagement metrics without improving understanding

Ethical Considerations of Dynamic Content Adaptation

- Transparency: students should know that content is being personalized and have some ability to see or influence the path chosen for them
- Avoiding a 'filter bubble' effect where a student is repeatedly steered toward narrow topics they already find easy, limiting broader skill development
- Fairness across demographics: the policy must be audited so it does not systematically provide lower-quality or lower-difficulty paths to particular student groups based on incidental correlations in the data
- Data privacy: behavioural and performance data used as state features must be protected, and used only for the stated educational purpose
- Avoiding over-optimization for short-term engagement at the expense of long-term learning outcomes, which requires the reward function to include delayed measures such as retention of knowledge over time

Illustrative Code (Python)

```
def learning_path_reward(quiz_score_gain, completion, time_spent,
                         retention_score, w1=2.0, w2=0.5, w3=1.5):
    reward = w1 * quiz_score_gain
    reward += w2 * completion
    reward += w3 * retention_score  # long-term mastery, not just clicks
    if time_spent > EXPECTED_TIME * 1.5:
        reward -= 0.5  # discourage filler/padding content
    return reward

def fairness_audit(recommendation_log, group_key):
    avg_difficulty = {}
```

```
    for rec in recommendation_log:
        g = rec[group_key]
        avg_difficulty.setdefault(g, []).append(rec["difficulty"])
    return {g: sum(v) / len(v) for g, v in avg_difficulty.items()}
```

Question 9: RL for Traffic Signal Control with Safety Constraints

Incorporating Constraints into the RL Model

- Emergency vehicle priority: the state includes a real-time emergency vehicle detection signal; when active, the action space is constrained (via action masking) so that only signal phases granting the emergency vehicle right-of-way are permitted, overriding the learned policy's normal preference
- Pedestrian safety: minimum pedestrian crossing time is enforced as a hard constraint on the action duration rather than left purely to reward-based learning, and a heavy negative reward is applied if a phase change would cut a crossing signal short
- General approach: this scenario is best handled as a Constrained MDP, where safety-critical rules are enforced as hard constraints (rule-based overrides or action masks) while the RL policy optimizes traffic flow within those constraints, rather than relying on reward penalties alone which could occasionally be violated

Interpretation of Impact on Congestion Reduction

With these constraints in place, the RL agent still substantially reduces average intersection wait time and congestion compared to fixed-timing signals, because it can adapt to real-time traffic density. However, the achievable congestion reduction is somewhat lower than an unconstrained optimum, since some green-time capacity is deliberately reserved for pedestrian crossings and emergency pre-emption. This trade-off is acceptable and necessary because safety objectives are given priority over pure throughput optimization.

Illustrative Code (Python)

```
def select_phase(state, policy, emergency_vehicle_present, min_ped_time_remaining):
    if emergency_vehicle_present:
        return emergency_priority_phase(state)  # hard override
    action = policy.select_action(state)
    if min_ped_time_remaining > 0:
        action = enforce_min_pedestrian_time(action, min_ped_time_remaining)
    return action

def traffic_reward(avg_wait_time_delta, emergency_delay, ped_violation):
    reward = -1.0 * avg_wait_time_delta
    if emergency_delay > 0:
        reward -= 20 * emergency_delay
    if ped_violation:
        reward -= 15
    return reward
```

Question 10: Traditional Optimization vs. Constrained RL for Manufacturing Scheduling

A manufacturing plant scheduling machines under resource limitations (machine capacity, labour, raw material) can be approached either with traditional optimization methods or with constrained RL.

Traditional Optimization Methods

- Examples: linear programming, mixed-integer programming, genetic algorithms, or heuristic dispatching rules

- Strength: given a well-defined, mostly static model of the plant, these methods can find a provably optimal or near-optimal schedule for that specific snapshot of demand and resources
- Weakness: they must typically be re-solved from scratch whenever conditions change (machine breakdown, urgent order, resource shortage), and they do not learn from experience across many operating scenarios

Constrained RL Approaches

- Strength: the agent learns a general scheduling policy from historical and simulated experience, so it can adapt in real time to breakdowns, demand changes, and resource fluctuations without being re-optimized from scratch each time
- Constraints (machine capacity limits, safety limits, order deadlines) are enforced through action masking or penalty terms so the learned policy respects operational limits
- Weakness: training requires significant data or a reliable simulator, convergence can be slower, and verifying strict guarantees of optimality/safety is harder than with exact mathematical optimization

Assessment of Adaptability

For dynamic production environments where conditions change frequently and unpredictably, constrained RL generally provides better adaptability, because it continuously improves its policy from ongoing experience rather than requiring a full re-optimization each time the environment changes. Traditional optimization remains preferable when the environment is relatively stable and an exact, verifiable optimum is required, or when there is insufficient data/simulation fidelity to train an RL agent reliably. In practice, many modern systems use a hybrid approach: traditional optimization for baseline scheduling and constrained RL for real-time adaptive adjustments.

Illustrative Code (Python)

```
from scipy.optimize import linprog

# Traditional optimization (LP) sketch: minimize makespan subject to resource limits
def traditional_schedule(cost_vector, constraint_matrix, resource_limits):
    result = linprog(c=cost_vector, A_ub=constraint_matrix, b_ub=resource_limits,
                     method="highs")
    return result.x

# Constrained RL sketch: penalize/mask resource-violating actions
def scheduling_reward(throughput_gain, resource_overuse, deadline_miss):
    reward = 2.0 * throughput_gain
    if resource_overuse > 0:
        reward -= 10 * resource_overuse  # constraint violation penalty
    if deadline_miss:
        reward -= 15
    return reward

def valid_machine_actions(state, capacity_limits, actions):
    return [a for a in actions if within_capacity(state, a, capacity_limits)]
```